COMPOSIÇÃO ALGORÍTMICA COM CADEIAS DE MARKOV

Lucas Momede Barreto Rezende
Universidade Federal de Minas Gerais
lucasmbr@ufmg.br

ABSTRACT

Este trabalho propõe um pipeline, em Python, para composição algorítmica utilizando Cadeias de Markov. Nesse sentido, foi utilizado como Corpus um conjunto de músicas (MIDI) que integram a soundtrack dos quatro primeiros jogos da franquia Silent Hill (Silent Hill 1, Silent Hill 2, Silent Hill 3 e Silent Hill 4: The Room). A restrição estilística se faz presente não somente pelas músicas fazerem parte da mesma franquia, mas também pela participação de um mesmo Sound Director, Akira Yamaoka, compositor principal e diretor de som da franquia nesses títulos [4].

1. INTRODUÇÃO

Para construir este projeto, a motivação partiu da oportunidade de criação de um corpus focado em *Silent Hill* utilizando arquivos do próprio jogo e na experimentação de como Cadeias de Markov se comportariam neste corpus. Nesse viés, dividiu-se o processo de implementação nas seguintes etapas:

1. Dada a escolha temática, compor o corpus.
2. Escolher uma forma de representação dos arquivos de entrada (MIDI) tal que fosse possível utilizá-los para gerar novas composições.
3. Decidir como aplicar cadeias de Markov na representação do MIDI.
4. Implementar geração e escolher hiperparâmetros modificáveis.
5. Comparar resultados (variações de hiperparâmetros).

2. TRABALHOS RELACIONADOS

O trabalho de Pachet [1] serviu como principal referência para as decisões de implementação. O autor descreve a geração musical a partir de padrões locais aprendidos no corpus, utilizando suas frequências para determinar probabilidades e escolhendo novos estados através de sorteios com peso. Essa ideia está diretamente relacionada ao processo utilizado neste trabalho para contar transições, calcular $P(x \mid c)$ e decidir o próximo estado durante a geração.

3. MÉTODO

3.1 Criação do Corpus

Em suma, foram coletadas músicas disponíveis em agregadores de MIDI online (MIDIFind, HomeTown e VGMusic [5–7]) e na extração e conversão de arquivos originais do jogo em MIDI (ver os repositórios sh2ex e kdttool [8, 9]). O corpus foi organizado em diferentes subconjuntos para facilitar os experimentos. A pasta `all`, utilizada como corpus principal, reúne 60 arquivos MIDI, sendo 10 extraídos dos arquivos originais de *Silent Hill 2* e 50 obtidos de MIDI disponibilizados online. Arquivos CSV em `_metadata/` registram a origem e o mapeamento das músicas. A estrutura completa do corpus e as instruções para sua reconstrução estão documentadas no `_metadata/CORPUS.md` do repositório.

3.2 Representação do MIDI

Para representar o MIDI adequadamente ao contexto, partiu-se da motivação dada por Trym Bø [3], onde ele discorre que a representação *piano-roll* é *"most structurally complex and detailed format"* e *"it seems to be the most on beat, and often uses silence to make cool beats or dynamic sounding melodies"*, em comparação com *event-based* e *relative pitch*. Note que este texto é empírico, mas serve como argumento para a escolha representativa.

Para leitura, pré-processamento e escrita dos arquivos MIDI, utilizou-se a biblioteca MusPy [2], escolhida por fornecer suporte direto à música simbólica e conversão para *piano-roll*. Assim, durante o pré-processamento, as *drum tracks* foram removidas, buscando simplificar, e baseando-se em Dong et al. [2], as informações de *velocity* foram ignoradas e a resolução temporal foi normalizada por um hiperparâmetro que define o número de passos por semínima. O MIDI é então convertido em um *piano-roll* binário de dimensão $T \times 128$, onde T representa o número de passos temporais e as 128 posições correspondem às possíveis notas MIDI. Silêncios internos são mantidos, enquanto silêncios no início ou no final da música são removidos.

Para reduzir a redundância da matriz, cada instante é representado apenas pelas notas ativas, resultando em:

$$S = (s_1, s_2, \ldots, s_T), \qquad s_t = (p_1, p_2, \ldots, p_k),$$

onde $p_i \in \{0, \ldots, 127\}$. Por exemplo,

$$[(60, 64, 67),\ (60, 64, 67),\ (),\ (62, 65, 69)],$$

onde notas na mesma tupla são simultâneas, repetições em instantes consecutivos representam duração e () representa silêncio.

3.3 Cadeia de Markov

Após a representação dos MIDI, cada música é interpretada como $S = (s_1, \ldots, s_T)$. Para uma Cadeia de Markov de ordem N, o próximo estado depende dos N estados anteriores:

$$P(s_t \mid s_{t-N}, \ldots, s_{t-1}).$$

As probabilidades são calculadas a partir das frequências observadas no corpus, contando quantas vezes cada estado s_t sucede um determinado contexto $s_{t-N}, \ldots, s_{t-1}$. Símbolos de início (<SOS>) e fim (<EOS>) são adicionados a cada música para que as transições não ocorram entre músicas distintas.

3.4 Geração e Hiperparâmetros

Após o treinamento, a geração inicia no símbolo *SOS*. A cada passo, o próximo estado é escolhido (sorteado) de acordo com as probabilidades de transição associadas ao contexto atual, repetindo o processo até que o símbolo *EOS* seja encontrado.

No que tange aos hiperparâmetros, considera-se a ordem N da Cadeia de Markov, que define quantos estados anteriores compõem o contexto, e a resolução temporal R, que define o número de passos por semínima. Para uma mesma configuração, diferentes seeds são utilizadas para gerar as músicas.

4. RESULTADOS

Para comparar os resultados dadas as variações de parâmetros, geraram-se 5 músicas para cada um desses valores: $N = \{1, 2, 3, 4, 5, 10, 15, 100\}$, $R = \{1, 2, 4, 8\}$. Ou seja, para cada combinação dos hiperparâmetros N e R, foram geradas 5 músicas utilizando diferentes seeds, que posteriormente foram comparadas qualitativamente por avaliação auditiva.

Começando pela variação de R (quantidade de passos por semínima), não se notou nenhuma mudança específica acarretada pela sua variação. A priori, durante testes rápidos na implementação, a redução de R parecia gerar músicas mais harmônicas, mas eventualmente, ao comparar várias músicas de mesmo N e seed, alterando somente o R, não se notou nenhuma diferença perceptível ao ouvido humano de quem testou auditivamente (apenas uma pessoa, então essa conclusão é mais subjetiva).

Agora, em relação à variação do hiperparâmetro N (memória de N estados anteriores), notaram-se diferenças no que tange à inovabilidade entre as músicas. Como esperado, valores de N mais altos causaram maiores repetições de trechos do corpus, sendo que, a partir de $N \geq 5$, isso se tornou gradativamente mais perceptível. Nesse sentido, as músicas geradas com N variando entre 1 e 4 proporcionaram melhor inovação, e mais especificamente o valor de $N = 2$ conseguiu, no geral, equilibrar criatividade e fidelidade ao corpus.

Interessante destacar que, para esse corpus especificamente, repetição de trechos específicos de outras músicas não é, necessariamente, um problema, visto que as músicas da franquia rotineiramente fazem referência umas às outras, vide o *leitmotiv* principal da série [10].

Por fim, como experimento adicional de timbre, algumas saídas foram renderizadas com o SoundFont *Silent Guy Vol. 2 PS2*, utilizando *Instruments PS2*. Os resultados com esses sons mostram como muito de uma identidade sonora ultrapassa o que uma reprodução com timbres genéricos consegue proporcionar, uma vez que os resultados foram auditivamente próximos à sonoridade do jogo.

Como exemplos dos resultados, foram selecionadas ao menos três músicas sintetizadas disponíveis em `outputs/favorites/`, no repositório. A nomenclatura dos arquivos segue o formato `markov_all_NXXX_RY_seedZZ`, onde `all` indica o corpus utilizado, N a ordem da Cadeia de Markov, R o número de passos por semínima e `seed` a semente aleatória utilizada na geração. Dentro de `outputs/favorites/`, estão os arquivos MIDI gerados em `.../MIDI/`, e as versões sintetizadas estão em `.../sintetizados_instruments_ps2/`. Foram incluídas mais de três músicas geradas.

5. DISCUSSÃO, LIMITAÇÕES E EXTENSÕES

A análise qualitativa mostrou que Cadeias de Markov são capazes de gerar músicas seguindo uma restrição estilística. Dito isso, este trabalho utilizou uma estrutura simples para representar as músicas MIDI, logo algumas informações que poderiam ser fulcrais para manter o estilo da restrição temática podem ter sido perdidas, como *velocity*, instrumentação e *drum tracks*, isto é, pode-se tentar validar melhor o trade-off entre simplicidade da estrutura e informações mantidas. Nesse sentido, essa é uma das principais limitações do projeto.

Outra limitação está na própria Cadeia de Markov, que modela bem padrões locais, mas possui dificuldade em representar estruturas musicais de longo prazo, como também discutido por Pachet [1]. Como extensões, seria possível testar Cadeias de Markov de ordem variável, outras formas de escolher o próximo estado e representações que preservem mais informações do MIDI. Além disso, métricas objetivas como polifonia e proporção de silêncio poderiam complementar a avaliação auditiva [2].

6. REFERENCES

[1] F. Pachet, "The Continuator: Musical Interaction With Style," *Journal of New Music Research*, vol. 32, no. 3, pp. 333–341, 2003.

[2] H.-W. Dong, K. Chen, J. McAuley, and T. Berg-Kirkpatrick, "MusPy: A Toolkit for Symbolic Music Generation," in *Proc. 21st Int. Society for Music Information Retrieval Conf. (ISMIR)*, Montréal, Canada, pp. 101–108, 2020.

[3] T. Bø, “Comparing MIDI Representations: The Battle Between Efficiency and Complexity,” *The MCT Blog*, Apr. 25, 2023. [Online]. Available: https://mct-master.github.io/machine-learning/2023/04/25/tryb-comparing-representations.html.

[4] GameFAQs, “Akira Yamaoka – Game Credits.” [Online]. Available: https://gamefaqs.gamespot.com/games/credit/2107-akira-yamaoka.

[5] MIDIFind, “Online Archive of Free MIDI Music Files.” [Online]. Available: https://midifind.com/.

[6] HomeTown, “Silent Hill – MIDI.” [Online]. Available: https://www.hometown.ru/midi.html.

[7] VGMusic, “Video Game Music Archive.” [Online]. Available: https://www.vgmusic.com/.

[8] Nisto, “sh2ex: Extracts Internal Filesystem and Sound Data of Silent Hill 2.” GitHub repository. [Online]. Available: https://github.com/Nisto/sh2ex.

[9] Nisto, “kdt-tool: General-Purpose Tool for Konami’s Sequenced Music Format KDT1.” GitHub repository. [Online]. Available: https://github.com/Nisto/kdt-tool.

[10] YouTube, “Silent Hill leitmotif.” [Online]. Available: https://www.youtube.com/watch?v=zn0-TYzBpwU.